1. **Aim/Overview of the practical:** Write a program based on Artificial neural. Also specify the data set used with number of input and output variables.

2. **Objective of the practical:** To build an Artificial Neural Network model that classifies iris flowers based on their physical measurements.

3. **Code for experiment/practical:**

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Load Wine dataset
wine = load_wine()
X = wine.data
y = wine.target.reshape(-1, 1)

# Normalize and encode
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y)

# ANN parameters
input_neurons = X.shape[1]   # 13 features
hidden_neurons = 8
output_neurons = 3
```

```
learning_rate = 0.01
epochs = 500

# Initialize weights
np.random.seed(0)
W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.zeros((1, hidden_neurons))
W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.zeros((1, output_neurons))

# Activation functions
def relu(x):
    return np.maximum(0, x)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# Training loop
for i in range(epochs):
    # Forward pass
    z1 = np.dot(X_scaled, W1) + b1
    a1 = relu(z1)
    z2 = np.dot(a1, W2) + b2
    a2 = softmax(z2)

    # Loss (Cross-Entropy)
    loss = -np.mean(y_encoded * np.log(a2 + 1e-8))

    # Backpropagation
    dz2 = a2 - y_encoded
    dW2 = np.dot(a1.T, dz2) / X_scaled.shape[0]
    db2 = np.sum(dz2, axis=0, keepdims=True) / X_scaled.shape[0]

    da1 = np.dot(dz2, W2.T)
    dz1 = da1 * (z1 > 0)
    dW1 = np.dot(X_scaled.T, dz1) / X_scaled.shape[0]
    db1 = np.sum(dz1, axis=0, keepdims=True) / X_scaled.shape[0]

    # Update weights
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
```

```
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if i % 100 == 0:
        print(f"Epoch {i}, Loss: {loss:.4f}")

# Prediction
z1 = np.dot(X_scaled, W1) + b1
a1 = relu(z1)
z2 = np.dot(a1, W2) + b2
a2 = softmax(z2)

pred = np.argmax(a2, axis=1)
actual = np.argmax(y_encoded, axis=1)
accuracy = np.mean(pred == actual)

print("\nDataset: Wine Classification")
print(f"Input Variables: {input_neurons}")
print(f"Output Classes: {output_neurons}")
print(f"Final Accuracy: {accuracy * 100:.2f}%")
```

4. Output:

```
Epoch 0, Loss: 2.5411
Epoch 100, Loss: 0.2716
Epoch 200, Loss: 0.1398
Epoch 300, Loss: 0.0971
Epoch 400, Loss: 0.0748

Dataset: Wine Classification
Input Variables: 13
Output Classes: 3
Final Accuracy: 94.38%
```

5. Learning Outcomes (What I have learnt):

- Understand how to implement a basic neural network using Python.
- Learn data preprocessing techniques before training an ANN.
- Analyze the model's performance and accuracy in classification tasks.